Lesson 12 Teacher's Key: Hearing It Happen

Godot Engine - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Introducing one shared AudioStreamPlayer2D (RoundSound) that swaps its .stream property between three interchangeable clips (win, wave, lose) rather than adding three separate always-idle nodes - a genuinely new Godot technique, and a nice illustration of reuse applied to a resource instead of a function.

Level 1 - Full Worked Solution

- RoundSound is new this week - a single node representing 'something happened that ends or advances the round,' regardless of which of the three outcomes it actually is.
- sfx_shoot and sfx_hit stay exactly where they were (Player's and Enemy's own AudioStreamPlayer2D nodes, from Lessons 3-4) - only the round-ending sounds get the new shared-node treatment, since shoot/hit can genuinely overlap each other while win/wave/lose are always mutually exclusive.
- Setting .stream immediately before every .play() call is the one detail that makes the shared-node approach work correctly - skipping it would replay whatever clip was assigned last.

Why This Structure

- A shared player for mutually-exclusive sounds (win/wave/lose) is a genuinely different pattern from Player's/Enemy's dedicated single-clip players (shoot/hit), and it's worth naming that difference explicitly: dedicated players suit sounds that can overlap themselves or each other; a shared swappable player suits outcomes where only one can ever be true.
- This is a natural, lightweight introduction to a 'sound manager' idea without building a whole new system - exactly the kind of small, justified new tool this course reaches for only when the old one stops fitting (the same discipline behind Lesson 9's real HUD and Lesson 10's reusable reset function).

Level 2 / Level 3 Talking Points

- Ask students to explain, in their own words, why RoundSound doesn't need its own is_playing-style guard the way Lesson 2's beep-beep blip did - because each event that triggers it can only happen once per moment (a wave clear, a true win, a loss), not something checked continuously every frame.

- For students comparing engines: Unity's roundAudioSrc.PlayOneShot() with three clips is conceptually identical to Godot's RoundSound.stream-swap this week, just using a different underlying mechanism (PlayOneShot doesn't need an explicit .stream assignment at all).

Common Student Mistakes

- Forgetting to set .stream before calling .play(), so RoundSound plays whatever it played last instead of the intended clip.
- Placing the SND_WAVE assignment/play() call outside the else branch, causing it to also fire on the true win.
- Wiring the new sound calls onto the wrong AudioStreamPlayer2D (e.g. reusing Player's shoot player for a round-ending sound), which technically works but breaks the 'dedicated vs. shared' distinction this lesson is teaching.